Aprendizaje Autónomo

Información General

Facultad:	FEIRNNET
Carrera:	Computación
Semestre y paralelo:	Ciclo 3-A
Periodo Académico:	Septiembre 2026-Marzo 2027
Estudiante:	Christopher Alexander Pineda Rodas
Enlace Ejercicios (GitHub/GitLab):	https://github.com/pinedarodaschris-oss/TrabajoAutonomoBaseDeDatos
Materia:	Base de datos
Docente Encargado:	Rene Guaman

Instrucciones

Para cada ejercicio se presenta la imagen del diagrama Entidad-Relación (notación de Chen) y el código MySQL del modelo relacional resultante.

Notación

Rectángulo: entidad (doble borde = entidad débil). Rombo: relación (doble rombo = relación identificadora). Óvalo: atributo (subrayado = clave; doble óvalo = multivaluado). Los números en rojo son las cardinalidades (1, N, M). El triángulo ISA indica la jerarquía de especialización.

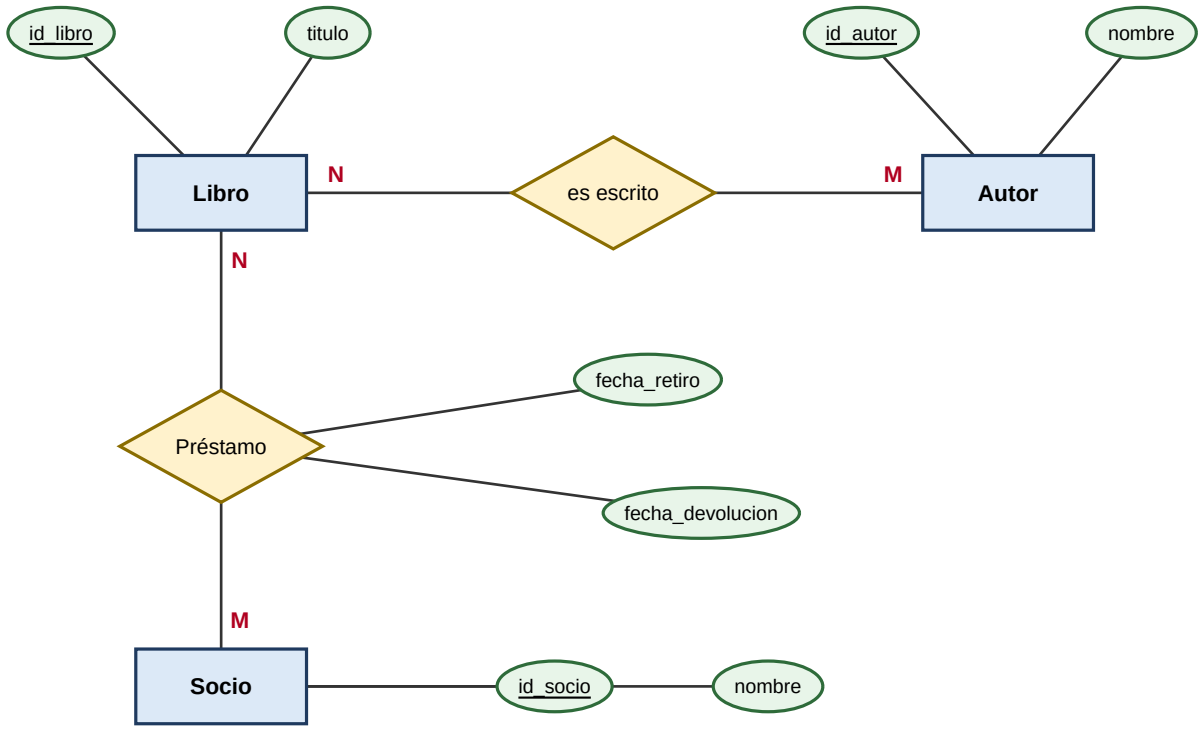
Nota sobre los diagramas

Se conserva la estructura de los diagramas dibujados a mano. Donde el enunciado no daba atributos se añadieron claves mínimas (id_*, nombre) para poder generar las tablas. Además se ajustaron algunos detalles para que el modelo coincida con el enunciado: en el Ejemplo 2 la relación Pedido-Plato es N:M con cantidad y precio_unitario; en el Ejemplo 3 dosis y duración pertenecen a la relación incluye y se agregó consultorio a Cita; en el Ejemplo 4 la reserva es N:M entre Vuelo y Pasajero; en el Ejercicio 5 la participación en eventos es N:M; en el Ejercicio 6 se añadieron precio (Producto) y fecha (compra); en el Ejercicio 8 se agregó la entidad Camión con su relación conduce.

Ejemplo 1: Sistema de Biblioteca

Una biblioteca registra libros y autores. Un libro puede tener varios autores y un autor varios libros. Los socios toman libros prestados; de cada préstamo interesan la fecha de retiro y la de devolución.

Diagrama Entidad-Relación



Código MySQL

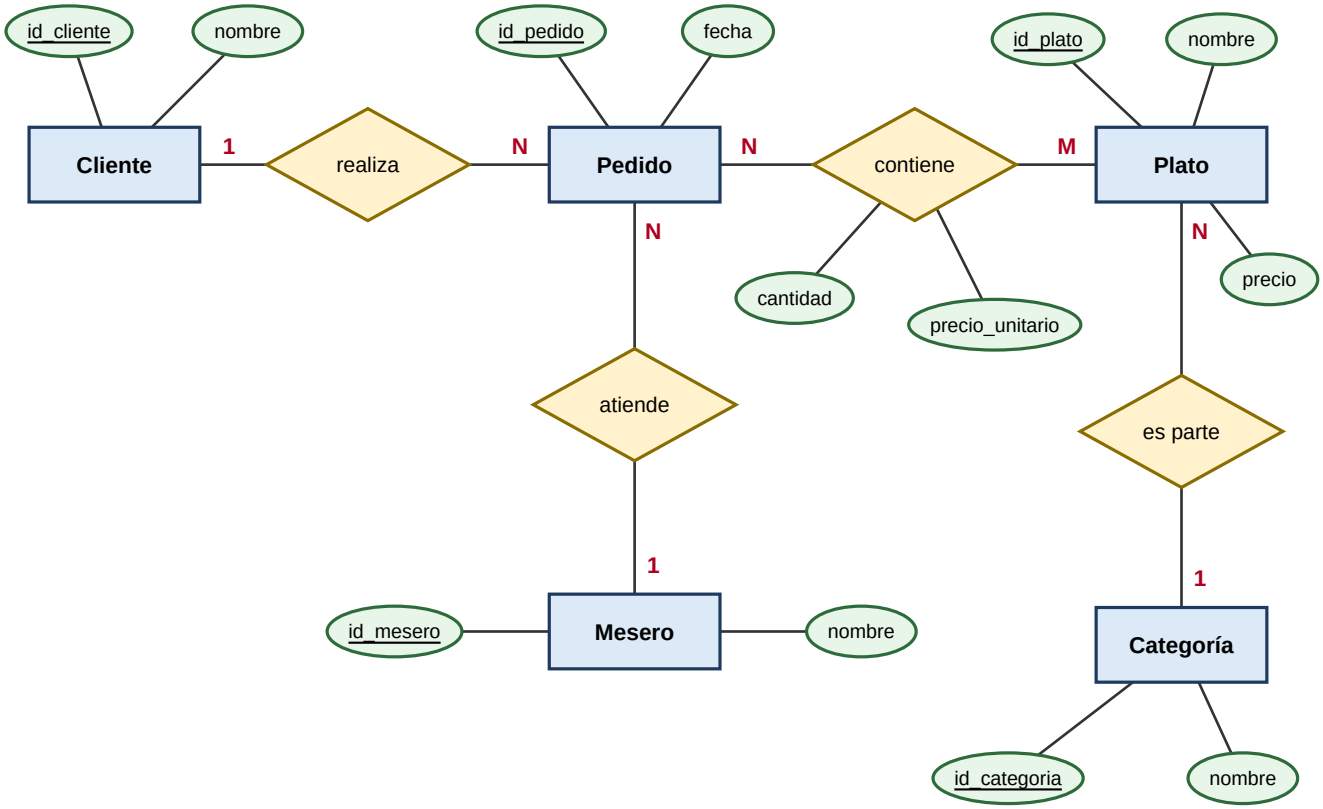
```
CREATE DATABASE IF NOT EXISTS biblioteca CHARACTER SET utf8mb4;
USE biblioteca;

CREATE TABLE libro (
  id_libro INT AUTO_INCREMENT PRIMARY KEY,
  titulo   VARCHAR(150) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE autor (
  id_autor INT AUTO_INCREMENT PRIMARY KEY,
  nombre   VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE socio (
  id_socio INT AUTO_INCREMENT PRIMARY KEY,
  nombre   VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

-- Relación N:M "es escrito"
CREATE TABLE libro_autor (
  id_libro INT NOT NULL,
  id_autor INT NOT NULL,
  PRIMARY KEY (id_libro, id_autor),
  FOREIGN KEY (id_libro) REFERENCES libro (id_libro),
  FOREIGN KEY (id_autor) REFERENCES autor (id_autor)
) ENGINE=InnoDB;

-- Relación N:M "Préstamo" con atributos propios
CREATE TABLE prestamo (
  id_prestamo      INT AUTO_INCREMENT PRIMARY KEY,
  id_libro         INT  NOT NULL,
  id_socio         INT  NOT NULL,
  fecha_retiro     DATE NOT NULL,
  fecha_devolucion DATE NULL,
  FOREIGN KEY (id_libro) REFERENCES libro (id_libro),
```

```
  FOREIGN KEY (id_socio) REFERENCES socio (id_socio),
  CHECK (fecha_devolucion IS NULL OR fecha_devolucion >= fecha_retiro)
) ENGINE=InnoDB;
```

Ejemplo 2: Pedidos de Restaurante

Los clientes realizan pedidos que contienen varios platos con distinta cantidad. Los platos se agrupan en categorías. Cada pedido lo atiende un único mesero. El precio del plato debe registrarse tal como estaba al momento del pedido, aunque luego cambie.

Diagrama Entidad-Relación



Código MySQL

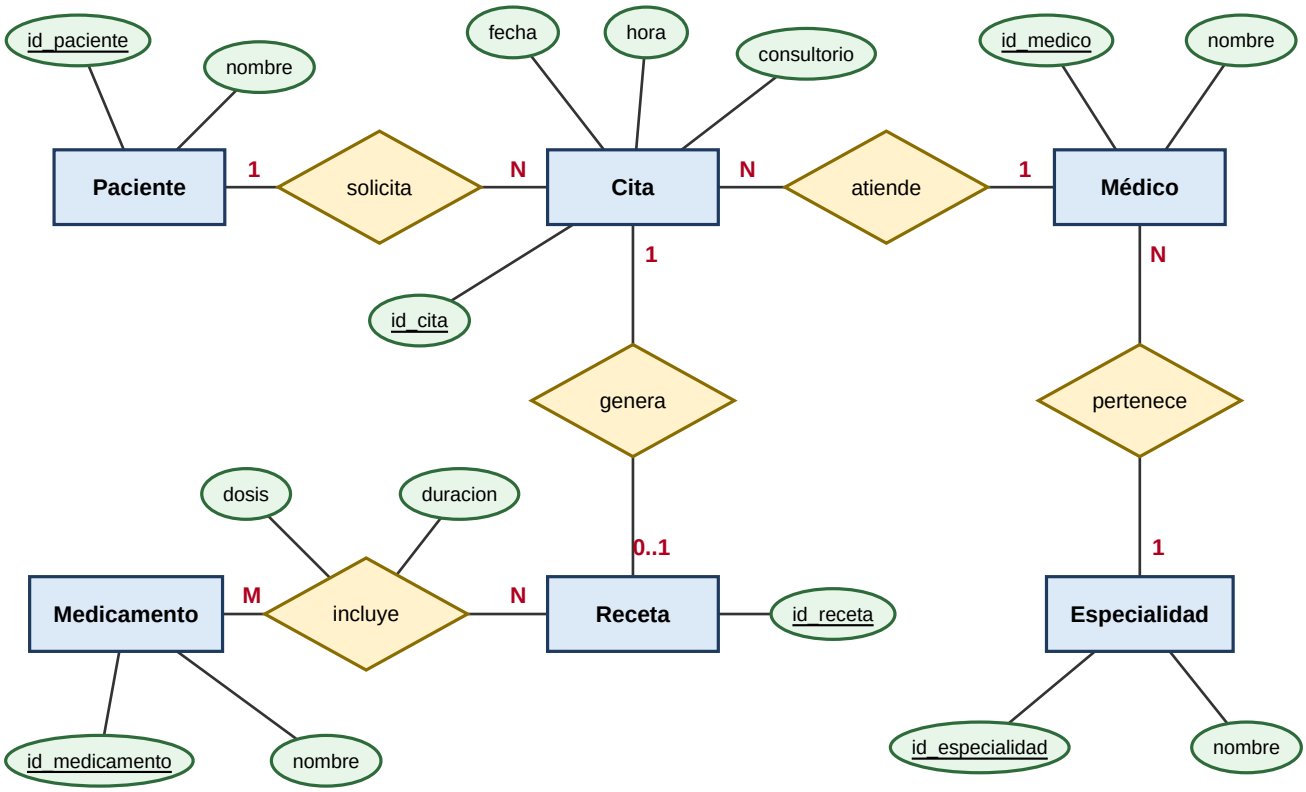
```
CREATE DATABASE IF NOT EXISTS restaurante CHARACTER SET utf8mb4;
USE restaurante;

CREATE TABLE cliente (
  id_cliente INT AUTO_INCREMENT PRIMARY KEY,
  nombre     VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE mesero (
  id_mesero INT AUTO_INCREMENT PRIMARY KEY,
  nombre    VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE categoria (
  id_categoria INT AUTO_INCREMENT PRIMARY KEY,
  nombre       VARCHAR(60) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE plato (
  id_plato     INT AUTO_INCREMENT PRIMARY KEY,
  nombre       VARCHAR(100)  NOT NULL,
  precio       DECIMAL(8,2)  NOT NULL,          -- precio vigente
  id_categoria INT NOT NULL,
  FOREIGN KEY (id_categoria) REFERENCES categoria (id_categoria)
) ENGINE=InnoDB;

CREATE TABLE pedido (
  id_pedido  INT AUTO_INCREMENT PRIMARY KEY,
  fecha      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  id_cliente INT NOT NULL,
  id_mesero  INT NOT NULL,                      -- un único mesero por pedido
  FOREIGN KEY (id_cliente) REFERENCES cliente (id_cliente),
  FOREIGN KEY (id_mesero)  REFERENCES mesero  (id_mesero)
```

```
) ENGINE=InnoDB;

-- Relación N:M "contiene": guarda el precio del momento del pedido
CREATE TABLE detalle_pedido (
  id_pedido       INT NOT NULL,
  id_plato        INT NOT NULL,
  cantidad        INT NOT NULL CHECK (cantidad > 0),
  precio_unitario DECIMAL(8,2) NOT NULL,
  PRIMARY KEY (id_pedido, id_plato),
  FOREIGN KEY (id_pedido) REFERENCES pedido (id_pedido),
  FOREIGN KEY (id_plato)  REFERENCES plato  (id_plato)
) ENGINE=InnoDB;
```

Ejemplo 3: Clínica Médica

Los pacientes solicitan citas; cada cita la atiende un médico a un paciente en fecha, hora y consultorio. De algunas citas se genera una receta que incluye medicamentos con dosis y duración. Cada médico pertenece a una especialidad.

Diagrama Entidad-Relación



Código MySQL

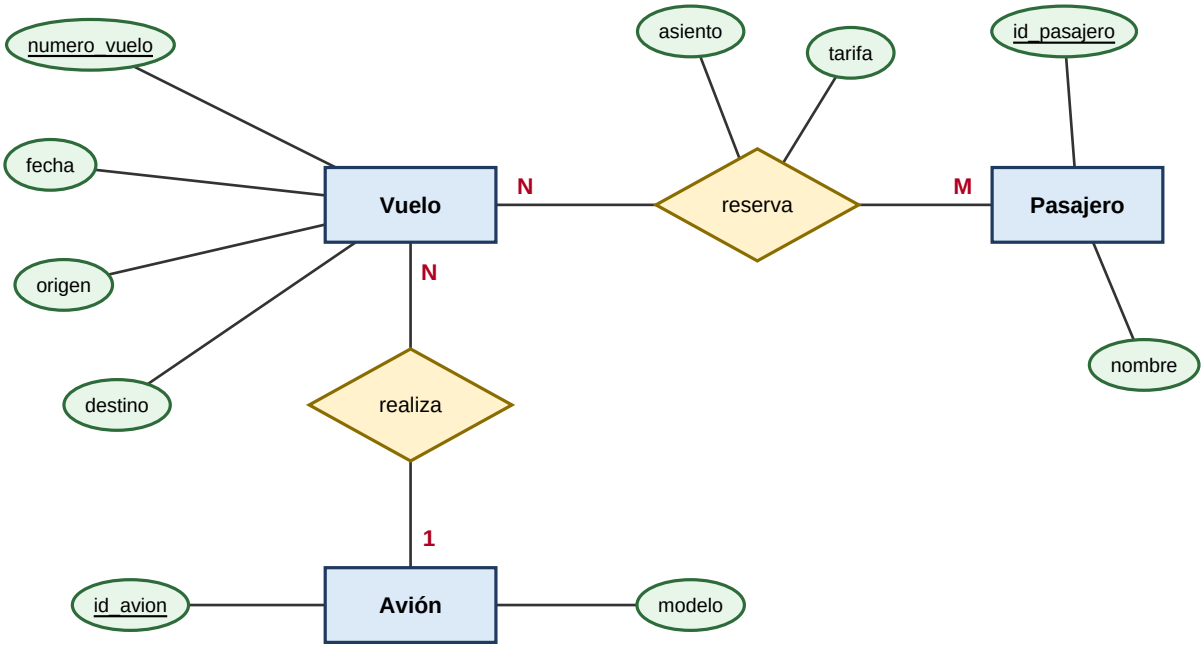
```
CREATE DATABASE IF NOT EXISTS clinica CHARACTER SET utf8mb4;
USE clinica;

CREATE TABLE especialidad (
  id_especialidad INT AUTO_INCREMENT PRIMARY KEY,
  nombre          VARCHAR(80) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE medico (
  id_medico       INT AUTO_INCREMENT PRIMARY KEY,
  nombre          VARCHAR(100) NOT NULL,
  id_especialidad INT NOT NULL,                 -- pertenece a una especialidad
  FOREIGN KEY (id_especialidad) REFERENCES especialidad (id_especialidad)
) ENGINE=InnoDB;

CREATE TABLE paciente (
  id_paciente INT AUTO_INCREMENT PRIMARY KEY,
  nombre      VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE cita (
  id_cita     INT AUTO_INCREMENT PRIMARY KEY,
  id_paciente INT NOT NULL,
  id_medico   INT NOT NULL,
  fecha       DATE NOT NULL,
  hora        TIME NOT NULL,
  consultorio VARCHAR(20) NOT NULL,
  FOREIGN KEY (id_paciente) REFERENCES paciente (id_paciente),
  FOREIGN KEY (id_medico)   REFERENCES medico   (id_medico),
  UNIQUE (id_medico, fecha, hora)               -- un médico no atiende dos citas a la vez
) ENGINE=InnoDB;

-- Una cita genera como máximo una receta (0..1)
```

```
CREATE TABLE receta (
  id_receta INT AUTO_INCREMENT PRIMARY KEY,
  id_cita   INT NOT NULL UNIQUE,
  FOREIGN KEY (id_cita) REFERENCES cita (id_cita)
) ENGINE=InnoDB;

CREATE TABLE medicamento (
  id_medicamento INT AUTO_INCREMENT PRIMARY KEY,
  nombre         VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

-- Relación N:M "incluye" con dosis y duración
CREATE TABLE receta_medicamento (
  id_receta      INT NOT NULL,
  id_medicamento INT NOT NULL,
  dosis          VARCHAR(60) NOT NULL,
  duracion       VARCHAR(60) NOT NULL,
  PRIMARY KEY (id_receta, id_medicamento),
  FOREIGN KEY (id_receta)      REFERENCES receta      (id_receta),
  FOREIGN KEY (id_medicamento) REFERENCES medicamento (id_medicamento)
) ENGINE=InnoDB;
```

Ejemplo 4: Sistema de Aerolínea

Una aerolínea opera vuelos (cada uno con número, fecha, origen y destino), cada vuelo lo realiza un avión. Los pasajeros realizan reservas para un vuelo específico, indicando asiento y tarifa pagada en el momento de la reserva.

Diagrama Entidad-Relación



Código MySQL

```
CREATE DATABASE IF NOT EXISTS aerolinea CHARACTER SET utf8mb4;
USE aerolinea;

CREATE TABLE avion (
  id_avion INT AUTO_INCREMENT PRIMARY KEY,
  modelo   VARCHAR(60) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE vuelo (
  id_vuelo     INT AUTO_INCREMENT PRIMARY KEY,
  numero_vuelo VARCHAR(10) NOT NULL,
  fecha        DATE        NOT NULL,
  origen       VARCHAR(60) NOT NULL,
  destino      VARCHAR(60) NOT NULL,
  id_avion     INT NOT NULL,                    -- cada vuelo lo realiza un avión
  FOREIGN KEY (id_avion) REFERENCES avion (id_avion),
  UNIQUE (numero_vuelo, fecha)
) ENGINE=InnoDB;

CREATE TABLE pasajero (
  id_pasajero INT AUTO_INCREMENT PRIMARY KEY,
  nombre      VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

-- Relación N:M "reserva": asiento y tarifa pagada al reservar
CREATE TABLE reserva (
  id_reserva  INT AUTO_INCREMENT PRIMARY KEY,
  id_vuelo    INT NOT NULL,
  id_pasajero INT NOT NULL,
  asiento     VARCHAR(5)    NOT NULL,
  tarifa      DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (id_vuelo)    REFERENCES vuelo    (id_vuelo),
  FOREIGN KEY (id_pasajero) REFERENCES pasajero (id_pasajero),
  UNIQUE (id_vuelo, asiento),                  -- un asiento por vuelo
  UNIQUE (id_vuelo, id_pasajero)
```

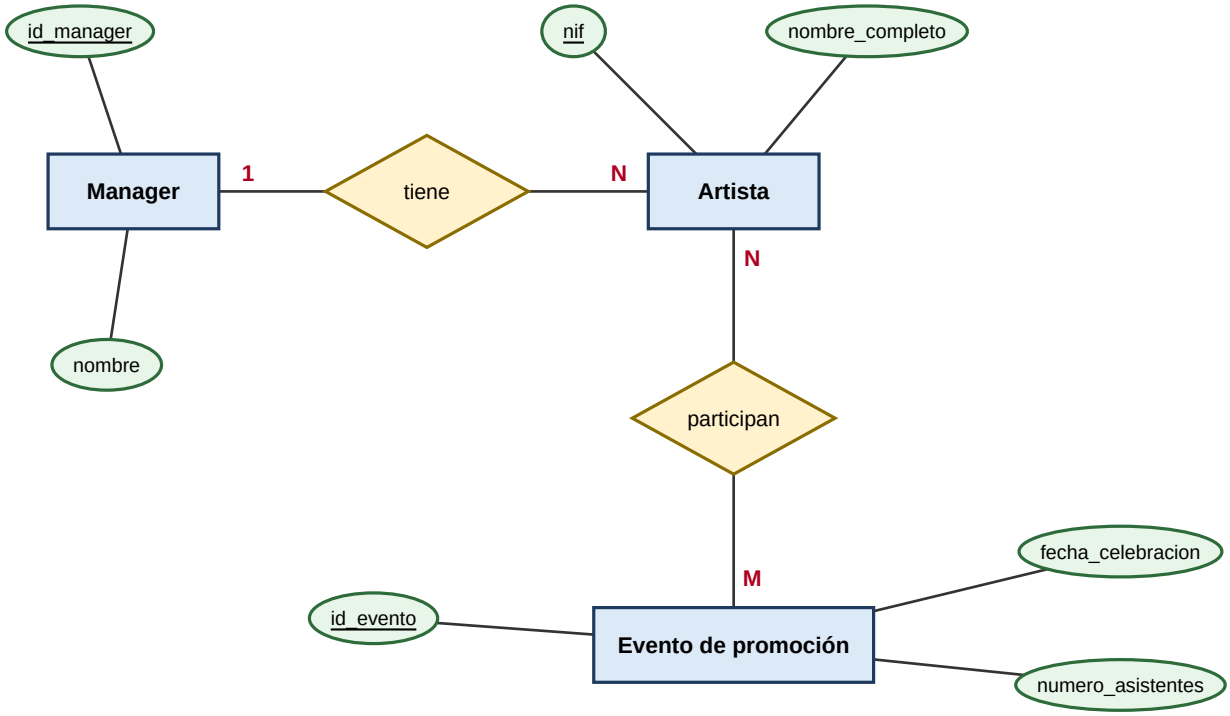
```
) ENGINE=InnoDB;
```

Ejercicio 5: Empresa Discográfica

Una empresa discográfica necesita modelar los datos sobre sus diferentes recursos según las siguientes características:

- Manager: de cada mánager se almacena un identificador de manager y su nombre. Un manager representa a una serie de artistas.
- Artista: un artista es representado por un manager. De los artistas se almacena su nombre completo (usando un único atributo) y su NIF.
- Evento de promoción: los artistas participan en eventos de promoción para dar a conocer sus trabajos. En un evento de promoción pueden participar varios artistas. De un evento se almacena un identificador único, la fecha de celebración y el número de asistentes.

Diagrama Entidad-Relación



Código MySQL

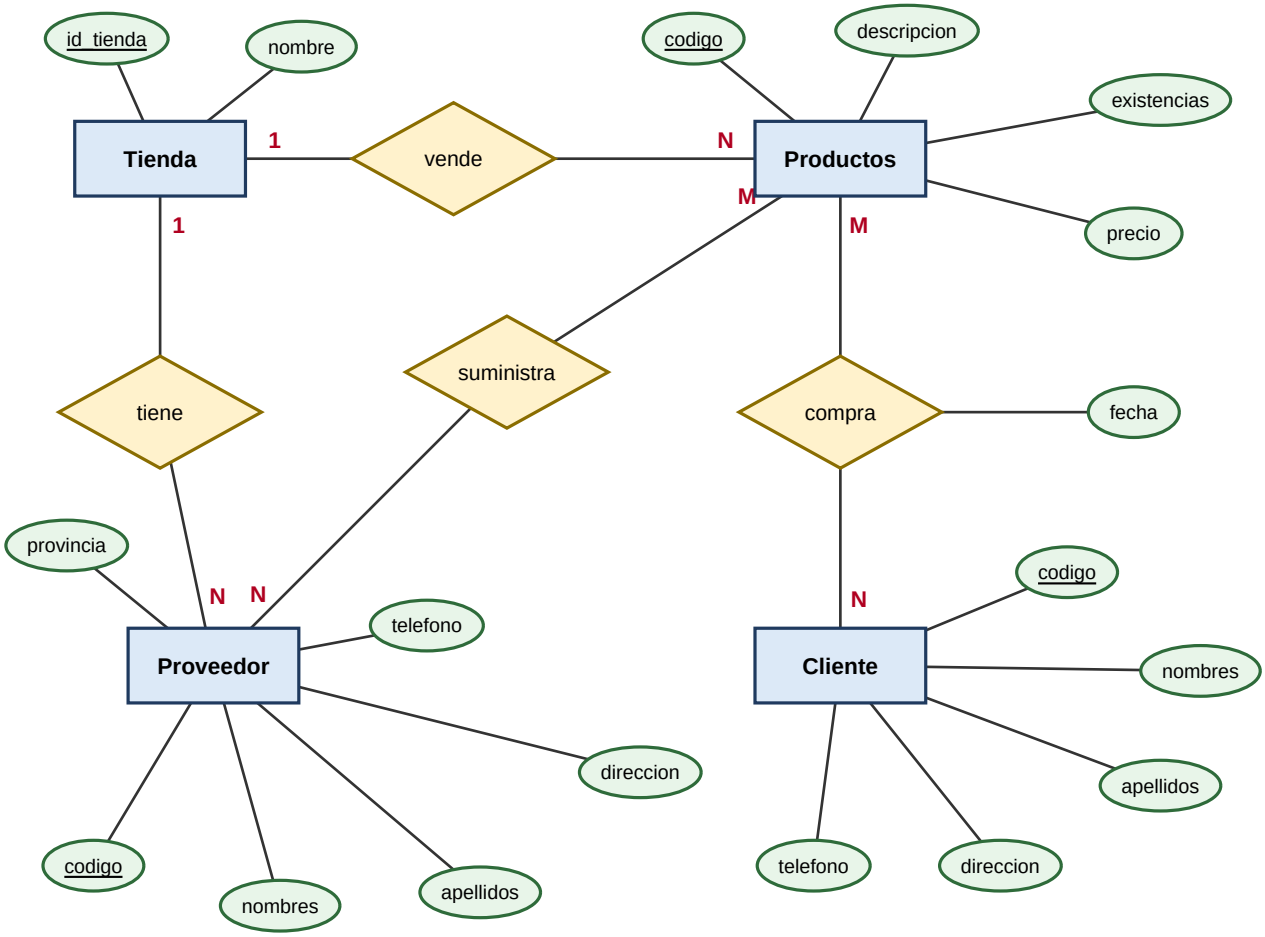
```
CREATE DATABASE IF NOT EXISTS discografica CHARACTER SET utf8mb4;
USE discografica;

CREATE TABLE manager (
  id_manager INT AUTO_INCREMENT PRIMARY KEY,
  nombre     VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE artista (
  nif             VARCHAR(15)  PRIMARY KEY,
  nombre_completo VARCHAR(120) NOT NULL,
  id_manager      INT NOT NULL,                 -- representado por un manager
  FOREIGN KEY (id_manager) REFERENCES manager (id_manager)
) ENGINE=InnoDB;

CREATE TABLE evento_promocion (
  id_evento         INT AUTO_INCREMENT PRIMARY KEY,
  fecha_celebracion DATE NOT NULL,
  numero_asistentes INT  NOT NULL DEFAULT 0
) ENGINE=InnoDB;

-- Relación N:M "participan"
CREATE TABLE participacion (
  nif       VARCHAR(15) NOT NULL,
  id_evento INT         NOT NULL,
  PRIMARY KEY (nif, id_evento),
```

```
  FOREIGN KEY (nif)       REFERENCES artista (nif),
  FOREIGN KEY (id_evento) REFERENCES evento_promocion (id_evento)
) ENGINE=InnoDB;
```

Ejercicio 6: Tienda Informática

Se desea crear una aplicación para la gestión de una tienda informática. La tienda dispone de una serie de productos que se venden a los clientes. De cada producto informático se desea guardar el código, descripción, precio y número de existencias. De cada cliente se desea guardar el código, nombre, apellidos, dirección y número telefónico. Un cliente puede comprar varios productos en la tienda y un mismo producto puede ser comprado por varios clientes. Cada vez que se compre un artículo quedará registrada la compra en la base de datos junto con la fecha en la que se compró el artículo. La tienda tiene contactos con varios proveedores que son los que suministran los productos. Un mismo producto puede ser suministrado por varios proveedores. De cada proveedor se desea guardar el código, nombres, apellidos, dirección, provincia y número de teléfono.

Diagrama Entidad-Relación



Código MySQL

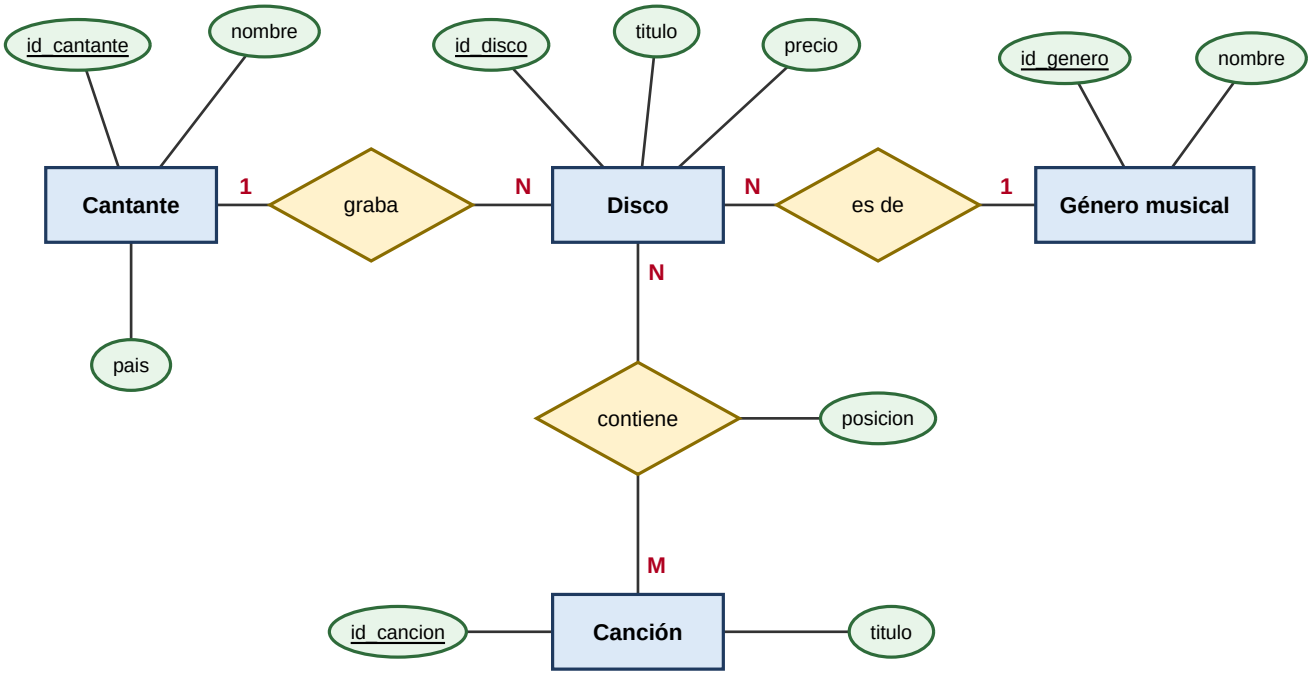
```
CREATE DATABASE IF NOT EXISTS tienda_informatica CHARACTER SET utf8mb4;
USE tienda_informatica;

CREATE TABLE tienda (
  id_tienda INT AUTO_INCREMENT PRIMARY KEY,
  nombre    VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE producto (
  codigo      VARCHAR(15) PRIMARY KEY,
  descripcion VARCHAR(150) NOT NULL,
  precio      DECIMAL(10,2) NOT NULL,
  existencias INT NOT NULL DEFAULT 0,
  id_tienda   INT NOT NULL,                    -- la tienda vende el producto
  FOREIGN KEY (id_tienda) REFERENCES tienda (id_tienda)
) ENGINE=InnoDB;

CREATE TABLE cliente (
  codigo    VARCHAR(15) PRIMARY KEY,
  nombres   VARCHAR(80)  NOT NULL,
```

```
  apellidos VARCHAR(80)  NOT NULL,
  direccion VARCHAR(150),
  telefono  VARCHAR(15)
) ENGINE=InnoDB;

CREATE TABLE proveedor (
  codigo    VARCHAR(15) PRIMARY KEY,
  nombres   VARCHAR(80)  NOT NULL,
  apellidos VARCHAR(80)  NOT NULL,
  direccion VARCHAR(150),
  provincia VARCHAR(60),
  telefono  VARCHAR(15),
  id_tienda INT NOT NULL,                       -- la tienda tiene contacto con el proveedor
  FOREIGN KEY (id_tienda) REFERENCES tienda (id_tienda)
) ENGINE=InnoDB;

-- Relación N:M "suministra"
CREATE TABLE suministra (
  codigo_proveedor VARCHAR(15) NOT NULL,
  codigo_producto  VARCHAR(15) NOT NULL,
  PRIMARY KEY (codigo_proveedor, codigo_producto),
  FOREIGN KEY (codigo_proveedor) REFERENCES proveedor (codigo),
  FOREIGN KEY (codigo_producto)  REFERENCES producto  (codigo)
) ENGINE=InnoDB;

-- Relación N:M "compra": cada compra queda registrada con su fecha
CREATE TABLE compra (
  id_compra       INT AUTO_INCREMENT PRIMARY KEY,
  codigo_cliente  VARCHAR(15) NOT NULL,
  codigo_producto VARCHAR(15) NOT NULL,
  fecha           DATE NOT NULL,
  FOREIGN KEY (codigo_cliente)  REFERENCES cliente  (codigo),
  FOREIGN KEY (codigo_producto) REFERENCES producto (codigo)
) ENGINE=InnoDB;
```

Ejercicio 7: Discos Musicales

- La tienda vende discos de diferentes géneros musicales y cantantes
- Cada género musical tiene un identificador
- Cada disco tiene título, género musical, precio y cantante
- De un cantante se registra el nombre y su país. Todo disco pertenece a un cantante y un cantante puede tener muchos discos. Además un disco tiene un conjunto de canciones (identificador y título)
- Un disco tiene muchas canciones y una canción puede estar en varios discos. Interesa conocer la posición de una canción en un determinado disco

Diagrama Entidad-Relación



Código MySQL

```
CREATE DATABASE IF NOT EXISTS discos_musicales CHARACTER SET utf8mb4;
USE discos_musicales;

CREATE TABLE genero_musical (
  id_genero INT AUTO_INCREMENT PRIMARY KEY,
  nombre    VARCHAR(60) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE cantante (
  id_cantante INT AUTO_INCREMENT PRIMARY KEY,
  nombre      VARCHAR(100) NOT NULL,
  pais        VARCHAR(60)  NOT NULL
) ENGINE=InnoDB;

CREATE TABLE disco (
  id_disco    INT AUTO_INCREMENT PRIMARY KEY,
  titulo      VARCHAR(120)  NOT NULL,
  precio      DECIMAL(8,2)  NOT NULL,
  id_genero   INT NOT NULL,
  id_cantante INT NOT NULL,                    -- todo disco pertenece a un cantante
  FOREIGN KEY (id_genero)   REFERENCES genero_musical (id_genero),
  FOREIGN KEY (id_cantante) REFERENCES cantante (id_cantante)
) ENGINE=InnoDB;

CREATE TABLE cancion (
  id_cancion INT AUTO_INCREMENT PRIMARY KEY,
  titulo     VARCHAR(120) NOT NULL
) ENGINE=InnoDB;
```

```
-- Relación N:M "contiene" con la posición de la canción en el disco
CREATE TABLE disco_cancion (
  id_disco   INT NOT NULL,
  id_cancion INT NOT NULL,
  posicion   TINYINT UNSIGNED NOT NULL,
  PRIMARY KEY (id_disco, id_cancion),
  UNIQUE (id_disco, posicion),
  FOREIGN KEY (id_disco)   REFERENCES disco   (id_disco),
  FOREIGN KEY (id_cancion) REFERENCES cancion (id_cancion)
) ENGINE=InnoDB;
```

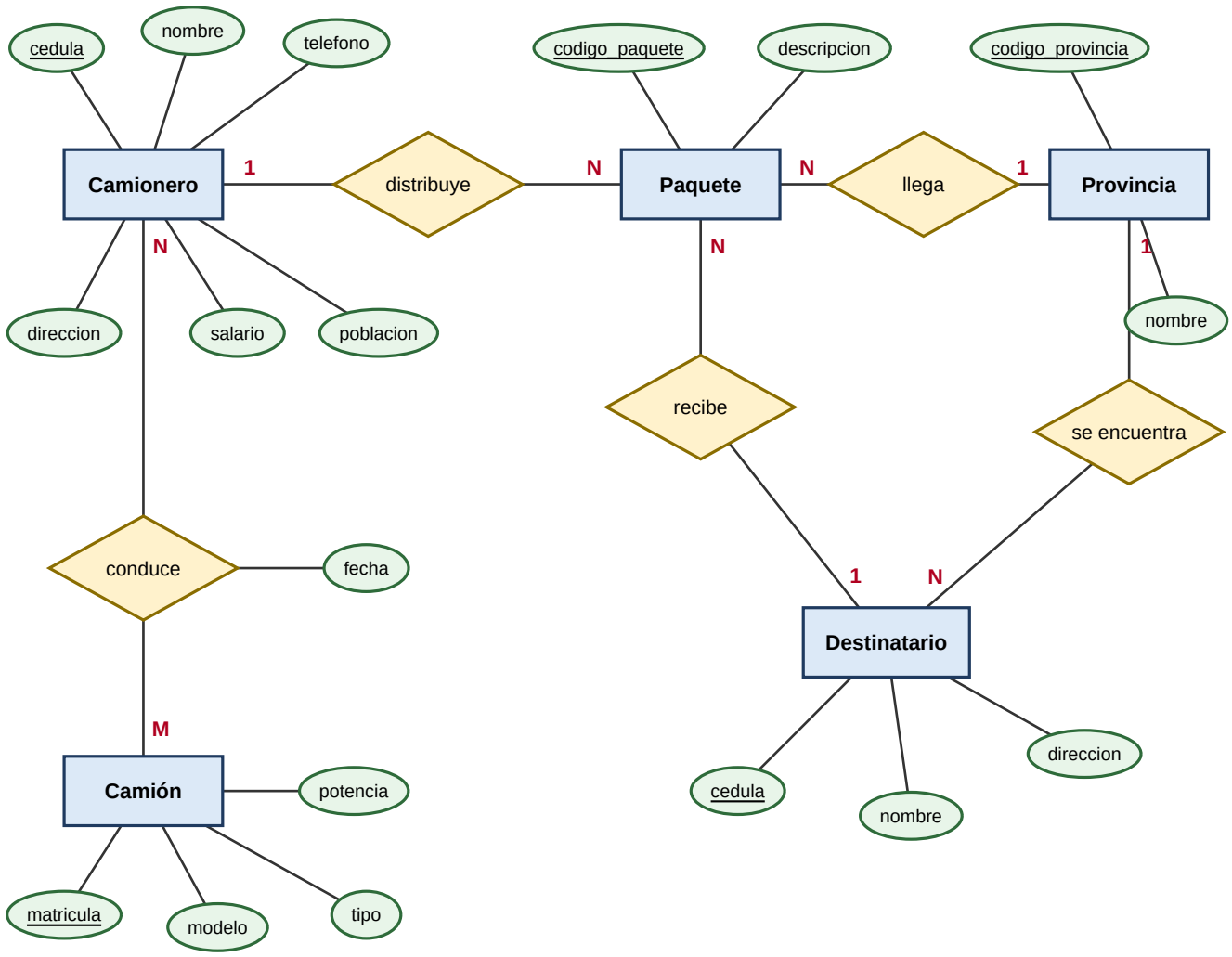
Ejercicio 8: Camiones

Se desea informatizar la gestión de una empresa de transporte que reparte paquetes por toda España. Los encargados de llevar los paquetes son los camioneros, de los que se quiere guardar la cédula, nombre, teléfono, dirección, salario y población en que vive. De los paquetes transportados interesa conocer el código del paquete, descripción, destinatario y dirección del destinatario.

Un camionero distribuye muchos paquetes, y un paquete sólo puede ser distribuido por un camionero. De las provincias a las que llegan los paquetes interesa guardar el código de provincia y el nombre. Un paquete sólo puede llegar a una provincia. Sin embargo, a una provincia pueden llegar varios paquetes.

De los camiones que llevan los camioneros, interesa conocer la matrícula, modelo, tipo y potencia. Un camionero puede conducir diferentes camiones en fechas diferentes, y un camión puede ser conducido por varios camioneros.

Diagrama Entidad-Relación



Código MySQL

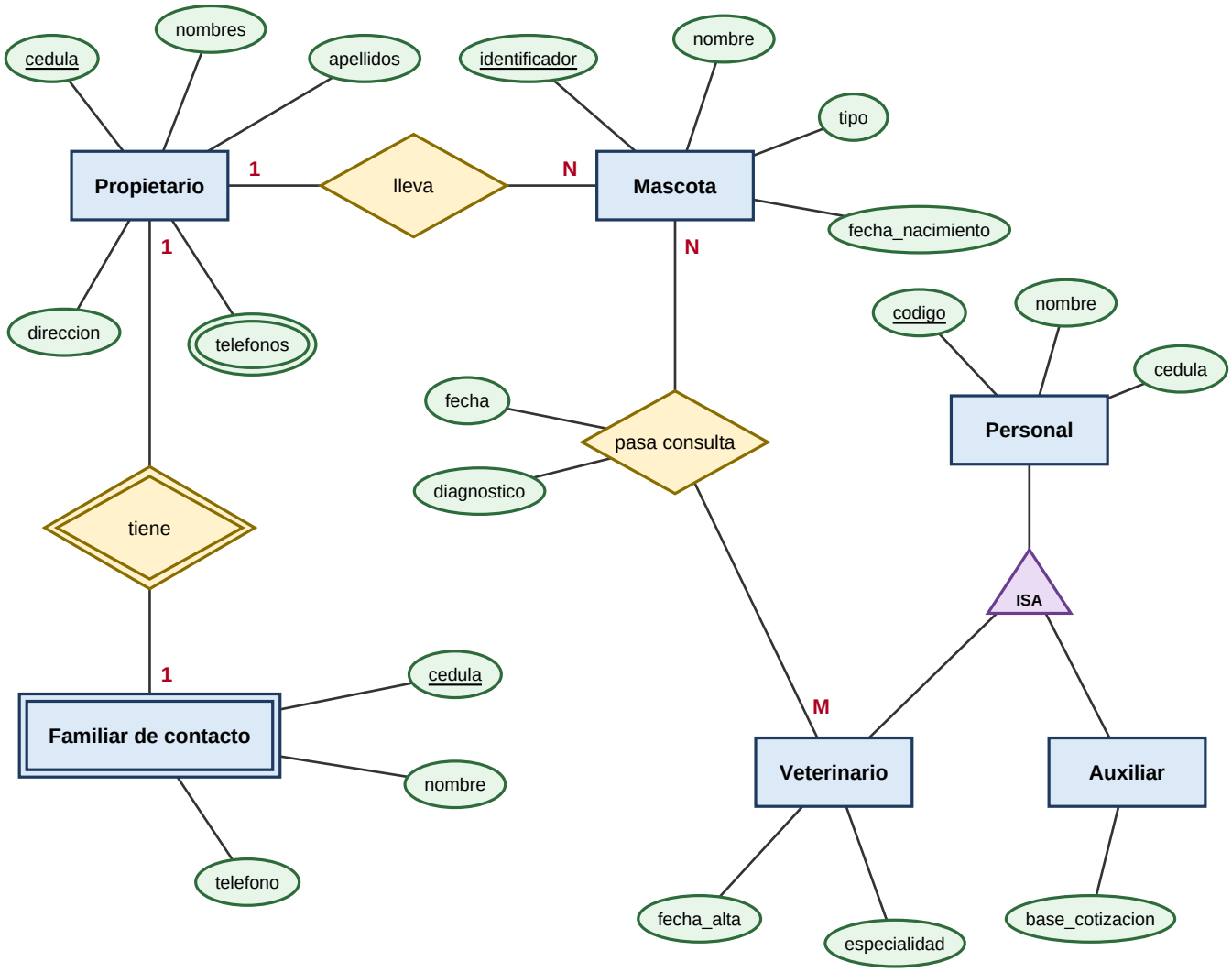
```
CREATE DATABASE IF NOT EXISTS transporte CHARACTER SET utf8mb4;
USE transporte;

CREATE TABLE camionero (
  cedula    VARCHAR(13) PRIMARY KEY,
  nombre    VARCHAR(100) NOT NULL,
  telefono  VARCHAR(15),
  direccion VARCHAR(150),
  salario   DECIMAL(10,2) NOT NULL,
  poblacion VARCHAR(80)
) ENGINE=InnoDB;

CREATE TABLE provincia (
  codigo_provincia VARCHAR(5) PRIMARY KEY,
  nombre           VARCHAR(60) NOT NULL
) ENGINE=InnoDB;
```

```
CREATE TABLE destinatario (
  cedula           VARCHAR(13) PRIMARY KEY,
  nombre           VARCHAR(100) NOT NULL,
  direccion        VARCHAR(150) NOT NULL,
  codigo_provincia VARCHAR(5)   NOT NULL,       -- "se encuentra" en una provincia
  FOREIGN KEY (codigo_provincia) REFERENCES provincia (codigo_provincia)
) ENGINE=InnoDB;

CREATE TABLE paquete (
  codigo_paquete      VARCHAR(15) PRIMARY KEY,
  descripcion         VARCHAR(150) NOT NULL,
  cedula_camionero    VARCHAR(13)  NOT NULL,    -- lo distribuye un solo camionero
  codigo_provincia    VARCHAR(5)   NOT NULL,    -- llega a una sola provincia
  cedula_destinatario VARCHAR(13)  NOT NULL,    -- lo recibe un destinatario
  FOREIGN KEY (cedula_camionero)    REFERENCES camionero    (cedula),
  FOREIGN KEY (codigo_provincia)    REFERENCES provincia    (codigo_provincia),
  FOREIGN KEY (cedula_destinatario) REFERENCES destinatario (cedula)
) ENGINE=InnoDB;

CREATE TABLE camion (
  matricula VARCHAR(10) PRIMARY KEY,
  modelo    VARCHAR(60) NOT NULL,
  tipo      VARCHAR(40) NOT NULL,
  potencia  INT NOT NULL
) ENGINE=InnoDB;

-- Relación N:M "conduce": un camionero conduce varios camiones en fechas distintas
CREATE TABLE conduce (
  id_conduccion    INT AUTO_INCREMENT PRIMARY KEY,
  cedula_camionero VARCHAR(13) NOT NULL,
  matricula        VARCHAR(10) NOT NULL,
  fecha            DATE NOT NULL,
  UNIQUE (cedula_camionero, matricula, fecha),
  FOREIGN KEY (cedula_camionero) REFERENCES camionero (cedula),
  FOREIGN KEY (matricula)        REFERENCES camion    (matricula)
) ENGINE=InnoDB;
```

Ejercicio 9: Veterinaria

- Del propietario interesa: cédula, apellidos, nombres, dirección, teléfonos
- De la mascota interesa: identificador, nombre, fecha de nacimiento, tipo
- Un propietario puede llevar una o varias mascotas
- Una mascota la lleva uno y solo un propietario
- Del personal de la clínica veterinaria interesa un código, la cédula, nombre
- Entre el personal de la clínica se distingue a los veterinarios y los auxiliares, pudiendo haber otros tipos
  - Del veterinario se guarda además su fecha_alta y especialidad
  - De los auxiliares interesa su base de cotización
- Las mascotas pasan consulta con los veterinarios. De manera que:
  - un veterinario puede pasar consulta a ninguna o a varias mascotas, y
  - una misma mascota de la clínica ha pasado consulta una o más veces en fechas diferentes con varios veterinarios.
  - Al pasar consulta se realiza un diagnóstico.
- Cada propietario tiene un familiar de contacto para casos de emergencia.
  - De este familiar interesa la cédula, nombre y teléfono.
  - Si el propietario se da de alta en la clínica veterinaria, el familiar ya no interesa.
  - El familiar es contacto de un solo propietario

Diagrama Entidad-Relación



Código MySQL

```
CREATE DATABASE IF NOT EXISTS veterinaria CHARACTER SET utf8mb4;
USE veterinaria;
```

```
CREATE TABLE propietario (
  cedula    VARCHAR(13) PRIMARY KEY,
  apellidos VARCHAR(80)  NOT NULL,
  nombres   VARCHAR(80)  NOT NULL,
  direccion VARCHAR(150)
) ENGINE=InnoDB;

-- Atributo multivaluado "telefonos"
CREATE TABLE propietario_telefono (
  cedula_propietario VARCHAR(13) NOT NULL,
  telefono           VARCHAR(15) NOT NULL,
  PRIMARY KEY (cedula_propietario, telefono),
  FOREIGN KEY (cedula_propietario) REFERENCES propietario (cedula)
    ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE mascota (
  id_mascota         INT AUTO_INCREMENT PRIMARY KEY,
  nombre             VARCHAR(60) NOT NULL,
  fecha_nacimiento   DATE,
  tipo               VARCHAR(40) NOT NULL,
  cedula_propietario VARCHAR(13) NOT NULL,      -- la lleva uno y solo un propietario
  FOREIGN KEY (cedula_propietario) REFERENCES propietario (cedula)
) ENGINE=InnoDB;

-- Entidad débil (1:1): si el propietario desaparece, el familiar ya no interesa
CREATE TABLE familiar_contacto (
  cedula_propietario VARCHAR(13) PRIMARY KEY,
  cedula             VARCHAR(13) NOT NULL UNIQUE,
  nombre             VARCHAR(80) NOT NULL,
  telefono           VARCHAR(15) NOT NULL,
  FOREIGN KEY (cedula_propietario) REFERENCES propietario (cedula)
    ON DELETE CASCADE
) ENGINE=InnoDB;

-- Superclase de la jerarquía ISA
CREATE TABLE personal (
  codigo VARCHAR(15) PRIMARY KEY,
  cedula VARCHAR(13) NOT NULL UNIQUE,
  nombre VARCHAR(80) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE veterinario (
  codigo       VARCHAR(15) PRIMARY KEY,
  fecha_alta   DATE        NOT NULL,
  especialidad VARCHAR(70) NOT NULL,
  FOREIGN KEY (codigo) REFERENCES personal (codigo) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE auxiliar (
  codigo           VARCHAR(15) PRIMARY KEY,
  base_cotizacion  DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (codigo) REFERENCES personal (codigo) ON DELETE CASCADE
) ENGINE=InnoDB;

-- Relación N:M "pasa consulta" (puede repetirse en fechas distintas)
CREATE TABLE consulta (
  id_consulta          INT AUTO_INCREMENT PRIMARY KEY,
  id_mascota           INT         NOT NULL,
  codigo_veterinario   VARCHAR(15) NOT NULL,
  fecha                DATE        NOT NULL,
  diagnostico          TEXT        NOT NULL,
  FOREIGN KEY (id_mascota)         REFERENCES mascota     (id_mascota),
  FOREIGN KEY (codigo_veterinario) REFERENCES veterinario (codigo)
) ENGINE=InnoDB;
```